



Universidad Mayor de San Andrés
Facultad de Ciencias Puras y Naturales



Facultad de Ciencias Puras y Naturales
FCPN
U.M.S.A.



I
N
F
O
R
M
Á
T
I
C
A



QUÍMICA



FÍSICA



ASTRONOMÍA
Y
ASTROFÍSICA

**Olimpiadas
Científicas
Escolares**



MATEMÁTICA



BIOLOGÍA



INFORMÁTICA

©Todos los derechos reservados, Facultad de Ciencias Puras y Naturales - UMSA

Universidad Mayor de San Andrés

Area de Informática

Nivel 1

Autores:

Unv. Rolando Troche

Unv. Gabriel Mazuelos

Revisión:

Msc. Aldo Valdéz

Msc. Jorge Terán

Índice general

1. Conceptos Básicos	1
1.1. Tipos de datos	1
1.2. Operadores	1
1.3. Comentarios	1
1.4. Lectura	2
1.5. Impresión	2
1.5.1. Ejercicios	3
2. Estructuras Condicionales	5
2.1. Introducción	5
2.1.1. Simples	5
2.1.2. Dobles	5
2.1.3. Múltiples	6
2.1.4. Ejercicios	7
3. Estructuras Repetitivas	9
3.1. Introducción	9
3.1.1. Ciclos FOR	9
3.1.2. Ciclos WHILE	9
3.1.3. Ejercicios	10
4. Problemas Ad Hoc	11
4.1. Introducción	11
4.2. Las Categorías	11
4.2.1. Juegos de Cartas	11
4.2.2. Ajedrez	11
4.2.3. Problemas relacionados con Palíndromos	11
4.2.4. Problemas relacionados a Anagramas	11
4.2.5. Problemas de la Vida Real	12
4.2.6. Problemas Ad Hoc con otras Temáticas	12
4.2.7. Ejercicios	12
5. Búsqueda Completa Iterativa	13

ii ÍNDICE GENERAL

5.1. Búsqueda completa	13
5.2. Búsqueda Completa Iterativa	13
5.3. Ejemplo	13
5.3.1. Ejercicios	14
6. Búsqueda Completa Recursiva	17
6.1. Búsqueda completa	17
6.2. Recursividad	17
6.2.1. Ejercicios	18
7. Sucesiones	19
7.1. Introducción	19
7.2. Tipos de sucesiones	19
7.2.1. Sucesiones aritméticas	19
7.2.2. Sucesiones geométricas	19
7.2.3. Ejercicios	20
8. Sucesión de Fibonacci	21
8.1. Iterativo	21
8.2. Recursivo	22
8.2.1. Ejercicios	22
9. Arreglos Unidimensionales	23
9.1. Introducción	23
9.1.1. Recorrido de un vector	23
9.1.2. Búsqueda de un elemento en un arreglo	23
9.1.3. Ejercicios	24
10. Ordenamiento de Arreglos	25
10.1. Ordenamiento por burbuja	25
10.1.1. Ordenamiento por inserción	25
10.1.2. Ordenamiento por selección	26
10.1.3. Ejercicios	27
11. Ordenamiento en C++	29
11.1. Introducción	29
11.1.1. Ordenamiento de un arreglo estático	29
11.1.2. Ordenamiento de un arreglo dinámico	29
11.1.3. Ejercicios	30
12. Permutaciones en C++	31

12.1. Introducción	31
12.1.1. Permutación de un arreglo estático	31
12.1.2. Permutación de un arreglo dinámico	32
12.1.3. Ejercicios	32
13. Números Primos	33
13.1. Introducción	33
13.1.1. Test de primalidad	33
13.1.2. Números primos mediante divisiones sucesivas	34
13.1.3. Ejercicios	34
14. Manejo y Procesamiento de Cadenas	35
14.1. Introducción	35
14.1.1. String en C++	35
14.1.2. Operación de concatenación	35
14.1.3. Métodos de la clase String en C++	36
14.1.4. Ejercicios	36
A. Lista de Ejercicios	37

1 Conceptos Básicos

1.1. Tipos de datos

Los tipos de datos básicos que se pueden encontrar son:

Tipo de Dato	bits	Limite inferior	Limite superior	Aproximado
bool	1	false	true	
char	8	-128	127	
int	32	-2147483648	2147483647	$\pm 2 \times 10^9$
long long	64	-9223372036854775808	9223372036854775807	$\pm 9 \times 10^{18}$
float	32	Numero de 32 bits con punto flotante		
double	64	Numero de 64 bits con punto flotante		

1.2. Operadores

Los operadores de asignación, comparación, y lógicos que se usan son:

Operacion	Sintaxis	Ejemplo
Asignacion	=	a=454
Igual	==	a==b
Distinto	!=	a!=b
Mayor(igual) que	> (>=)	a>b (a>=b)
Menor(igual) que	< (<=)	a<b (a<=b)
and	&&	a==b && a>0
or		a!=0 b>0
not	!	!(a==b)

1.3. Comentarios

Los comentarios en un código son líneas que no se tomaran en cuenta al momento de compilar, y donde usualmente colocaremos ayudas y/o explicaciones de lo que hacemos para facilitar la lectura del código. Los comentarios son líneas que comienzan con // o si queremos comentar un bloque se debe usar /* para comenzar y */ para finalizar

```
1 //Estos es un comentario de una sola linea
2 /*
3 Esto es un comentario
```

2 Capítulo 1 *Conceptos Básicos*

```
4   en varias líneas
5   */
```

1.4. Lectura

Para leer datos desde el teclado es necesario usar la librería *iostream* o *stdio.h*.

```
1  #include <iostream>; //Para agregar la libreria iostream a nuestro
   programa
2  #include <string> //Libreria para el uso de cadenas
3  using namespace std;
4  int main () { //Metodo principal (main)
5      int n;
6      cin>>n;    //Leer un numero entero de 32 bits
7      float f;
8      cin>>f;    //Leer un numero con punto flotante
9      double d;
10     cin>>d;    //Leer un numero de 64 bits con punto flotante
11     long long l;
12     cin>>l;    //Leer un numero entero de 64 bits
13     String s;
14     cin>>s;    //Lee solo hasta el primer espacio.
15     String s1;
16     getline(cin, s1); //Lee hasta el salto de linea (\n)
17     while(cin>>n){    //Leemos hasta fin de archivo (mientras exista mas
                        datos por leer)
18     }
19 }
```

1.5. Impresion

Para mostrar datos en la pantalla es necesario usar la librería *iostream* o *stdio.h*.

```
1  #include <iostream>
2  #include <stdio.h> //Necesario para usar printf
3  using namespace std;
4  int main () {
5      cout<<endl; //Imprimir salto de linea
6      cout<<" "<<n<< //Para imprimir texto de forma literal se coloca entre
                        comillas () y una variable acontinuacion
7      printf("*formato*",<variables*>); //Impresion con formato
8  }
```

En **formato** se especifica como se imprimiran las variables, y en **variables** cuales se han de imprimir, se debe colocar las variables en el orden que aparecen en **formato**, en formato se usan los siguientes identificadores para las variables:

Identificador	Tipo de variable
%c	Caracter (char)
%s	Cadenas (String)
%d	Entero (int)
%ld	Entero largo (long)
%f	Numero con punto flotante (float)
%lf	Numero flotante de doble preciciosn (double)
%. 'x' lf	Imprimir con 'x' decimales de presicion

```

1  #include <stdio.h>
2  #include <string>
3  using namespace std;
4  int main() {
5      int a=23;
6      long long b=23123;
7      char c='a';
8      float f=1231.4534;
9      double d=4564.1315;
10     printf("%d %lld %f %c %f %.2f",a ,b ,f, c, d, f);
11 }
```

1.5.1. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1032	Hola Mundo!	Impresion	https://jv.umsa.bo/problem.php?id=1232
1425	Imprimir PI E Au	Impresion	https://jv.umsa.bo/problem.php?id=1425

2

Estructuras Condicionales

2.1. Introducción

Las estructuras condicionales sirven para comparar valores de variables, y en base a estas comparaciones tomar decisiones. Estas pueden ser simples, dobles o múltiples.

2.1.1. Simples

Esta clase de estructura condicional, es la que cuando se realiza la comparación, tiene la característica de que solo se sigue un conjunto de instrucciones, siempre y cuando la comparación tenga como resultado el valor lógico de **VERDAD**.

Algorithm 1 Condicionales simples

```
1: procedure CONDICIONAL( $a, b$ )  
2:   if  $a > b$  then  
3:     print ( $a$ )  
4:   end if  
5: end procedure
```

```
1  #include <bits/stdc++.h>  
2  
3  using namespace std;  
4  
5  int main(){  
6      int a = 10, b = 2;  
7      if(a > b){  
8          cout<<a<<endl;  
9      }  
10     return 0;  
11 }
```

2.1.2. Dobles

Esta comparación nos permite utilizar un bloque de instrucciones que se van a ejecutar si la condición es **VERDADERA** y otro bloque de instrucciones que se ejecutan si la condición es **FALSA**.

Algorithm 2 Condicionales dobles

```

1: procedure CONDICIONAL( $a, b$ )
2:   if  $a > b$  then
3:     print ( $a$ )
4:   else
5:     print ( $b$ )
6:   end if
7: end procedure

```

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      int a = 10, b = 2;
7      if(a > b){
8          cout<<a<<endl;
9      }else{
10         cout<<b<<endl;
11     }
12     return 0;
13 }

```

2.1.3. Múltiples

Este tipo de comparación, nos permite comparar un valor simultáneamente con otros valores y ejecutar un bloque de instrucciones cuando la comparación sea **VERDADERA**.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      int a = 4;
7      switch(a){
8          case 1:
9              cout<<"uno"<<endl;
10             break;
11          case 2:
12              cout<<"dos"<<endl;
13             break;
14          case 3:

```

```
15         cout<<"tres"<<endl;
16         break;
17     case 4:
18         cout<<"cuatro"<<endl;
19         break;
20     case 5:
21         cout<<"cinco"<<endl;
22         break;
23
24     }
25     return 0;
26 }
```

2.1.4. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1226	Temperatura	Condicionales	https://jv.umsa.bo/problem.php?id=1226

3

Estructuras Repetitivas

3.1. Introducción

Las estructuras repetitivas nos sirven para poder ejecutar varias veces un bloque de instrucciones.

3.1.1. Ciclos FOR

La instrucción **FOR** nos permite realizar iteraciones en un rango determinado y la siguiente es su sintaxis:

Algorithm 3 Ciclo For

```
1: procedure NUMERACION
2:   for  $i$  in range (1,10) do
3:     print ( $i$ )
4:   end for
5: end procedure
```

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      for(int i = 1; i <= 10; i++){
7          cout<<i<<endl;
8      }
9      return 0;
10 }
```

3.1.2. Ciclos WHILE

A diferencia de un ciclo **FOR**, el ciclo **WHILE** nos permite iterar un bloque de instrucciones, mientras algunas condiciones sean verdaderas. Por ejemplo, a continuación pasaremos a contar el numero de dígitos que tiene un numero natural N.

Algorithm 4 While

```

1: procedure CONTAR DIGITOS( $N$ )
2:    $resultado \leftarrow 0$ 
3:   while  $N \neq 0$  do
4:      $N \leftarrow N/10$ 
5:      $resultado \leftarrow resultado + 1$ 
6:   end while
7:   print ( $resultado$ )
8: end procedure

```

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      int N = 4324;
7      int resultado = 0;
8      while(n > 0){
9          n = n / 10;
10         resultado = resultado + 1;
11     }
12     cout<<resultado<<endl
13     return 0;
14 }

```

3.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1248	Números Armónicos	Ciclos For	https://jv.umsa.bo/problem.php?id=1248
1249	Primeros números	Ciclos For	https://jv.umsa.bo/problem.php?id=1249

4

Problemas Ad Hoc

4.1. Introducción

De acuerdo a USACO los problemas Ad Hoc son problemas que "no se pueden clasificar en ningún otro lugar" dado que todas las descripciones de problemas y soluciones son **únicas**. Muchísimos de los problemas Ad Hoc son fáciles de resolver, pero esto no aplica a todos los problemas Ad Hoc.

Existen muchos problemas Ad Hoc, la idea de estos es que se resuelvan sin el uso de estructuras de datos avanzadas o algoritmos. Muchos de estos problemas Ad Hoc tienen soluciones **fáciles** pero algunos otros unas mas **difíciles**. [?]

4.2. Las Categorías

4.2.1. Juegos de Cartas

Existen muchos problemas Ad Hoc que involucran juegos populares, por eso se puede ver algunos problemas que tienen que ver con el uso de cartas.

4.2.2. Ajedrez

Otro juego muy popular es el ajedrez, y se pueden ver problemas que involucren tanto las piezas, como el tablero o ambos.

4.2.3. Problemas relacionados con Palíndromos

Estos son otros problemas clásicos. Una palabra u oración se dice palíndroma si esta se lee de igual manera en ambos sentidos. Ej: ABBBA. Una técnica fácil para ver si una palabra es palíndroma es recorrer esta desde el principio hasta el medio y verificar si cada letra tiene su correspondiente desde el otro sentido.

4.2.4. Problemas relacionados a Anagramas

Un anagrama es una palabra u oración cuyas letras pueden ser reordenadas de alguna manera, tal que se obtiene otra palabra u oración. Ej: palabraA = "GATO" y palabraB = "TOGA", una técnica sencilla para verificar si una palabra es Anagrama de otra, es ordenando ambas y comparándolas; en este caso palabraA = "AGTO" y palabraB = "AGTO".

4.2.5. Problemas de la Vida Real

Este es uno de los tipos de problemas, más comunes en los Ad Hoc. Y pueden ser, desde los muy fáciles hasta los increíblemente difíciles y/o tediosos. Estos describen situaciones de la vida diaria, este tipo de problemas ayudan bastante a comprender que la historia de un problema, puede llegar a ser muy importante y se debe leer con cuidado ya que esta puede contener detalles sumamente valiosos.

4.2.6. Problemas Ad Hoc con otras Temáticas

Existen problemas Ad Hoc para casi cualquier cosa, por lo que es muy común ver este tipo de problemas en otras temáticas, como ser:

- Problemas con arreglos unidimensionales (vectores)
- Problemas con cálculos matemáticos
- Problemas con manejo de cadenas
- Problemas de geometría básica

4.2.7. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1065	Kuadraconia	AdHoc	https://jv.umsa.bo/problem.php?id=1065

Juez Patito	Nombre	Tipo de problema	Enlace
1065	Kuadraconia	Adhoc	https://jv.umsa.bo/problem.php?id=1065

5

Búsqueda Completa Iterativa

5.1. Búsqueda completa

La técnica de búsqueda completa, también conocida como técnica de fuerza bruta, es la que, tal como su nombre indica, realizará una búsqueda por todo el espacio de solución posible, es decir, que verificará todas las posibles soluciones para encontrar la que necesitemos. Muchas veces se puede optar por una solución con búsqueda completa, cuando nos aseguramos que esta, entra en tiempo, y/o no hay otra mejor solución. Una buena solución con búsqueda completa nunca debería fallar, pero si puede ser demasiado lenta para el tiempo que tenemos, por lo que es necesario saber reconocer cuando es una buena idea utilizar esta técnica

5.2. Búsqueda Completa Iterativa

Esta técnica se caracteriza por el uso de varios *FOR* para el recorrido de todas las posibles soluciones, es común usarlo, por ejemplo, en problemas que buscan combinaciones óptimas o que cumplan determinadas características. En el contexto de los algoritmos de búsqueda completa, existe lo que denomina **podar**, que significa, decidir no explorar cierto espacio de las soluciones posibles, ya que se puede demostrar que es imposible hallar la solución en ese lugar, esta idea es muy utilizada ya que ayuda a mejorar ciertos tiempos de ejecución. Una de sus desventajas, es que dependiendo del problema, depende de que tan bueno es uno con las soluciones analíticas, el entender el problema y sus restricciones y ver como una "búsqueda completa" puede reducirse en ciertos puntos.

5.3. Ejemplo

Te dan 3 números enteros A, B, C ($1 \leq A, B, C \leq 10000$), tu tarea es encontrar otros 3 números enteros (diferentes) tales que se cumpla las siguientes condiciones: $x + y + z = A$, $x \times y \times z = B$ y $x^2 + y^2 + z^2 = C$. Una primera solución puede ser probar todos los posibles enteros entre $[-10000, 10000]$ y ver para cuales de estos se cumplen las restricciones, esto nos dejaría una solución así:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main(){
4      int A,B,C;
5      cin>>A>>B>>C;
6      for (int x=-10000; x<10000; ++x) {
```

```

7         for (int y=-10000; y<10000; ++y) {
8             for (int z=-10000; z<10000; ++z) {
9                 if(x+y+z==A and x*y*z==B and x*x+y*y+z*z==C)
10                    cout<<x<<" "<<y<<" "<<z<<"\n";
11             }
12         }
13     }
14     return 0;
15 }

```

Bien pues es obvio ver que esta solución es muy lenta ya que tiene complejidad $O(20000^3)$, entonces podemos "podar" nuestro espacio de posibles soluciones para mejorar esto. Podemos partir de la tercera ecuación $x^2 + y^2 + z^2 = C$, si tomamos que C tiene su valor más alto 10000 y damos a $x = 1$ e $y = 2$ ya que no pueden ser iguales, tenemos que los posibles valores que puede tomar z son $[-100, 100]$, tomando en cuenta este razonamiento y haciendo de manera similar con x e y , podemos ver que basta con buscar en ese intervalo, entonces nos quedaría:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int main(){
4      int A,B,C;
5      cin>>A>>B>>C;
6      for (int x=-100; x<=100; ++x) {
7          for (int y=-100; y<=100; ++y) {
8              for (int z=-100; z<=100; ++z) {
9                  if(x!=y and y!=z and z!=x and x+y+z==A and x*y*z
10                     ==B and x*x+y*y+z*z==C)
11                     cout<<x<<" "<<y<<" "<<z<<"\n";
12              }
13          }
14      }
15      return 0;
16 }

```

Con esto la solución se reduce bastante. ¿Puedes encontrar otra manera de reducir el espacio de la solución aún más?

5.3.1. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

5.3 Ejemplo 15

Juez Patito	Nombre	Tipo de problema	Enlace
1038	La clase estaba aburrida	Busqueda Completa	https://jv.umsa.bo/problem.php?id=1038

6

Búsqueda Completa Recursiva

6.1. Búsqueda completa

La técnica de búsqueda completa, también conocida como técnica de fuerza bruta, es la que, tal como su nombre indica, realizará una búsqueda por todo el espacio de solución posible, es decir, que verificará todas las posibles soluciones para encontrar la que necesitemos. Muchas veces se puede optar por una solución con búsqueda completa, cuando nos aseguramos que esta, entra en tiempo, y/o no hay otra mejor solución. Una buena solución con búsqueda completa nunca debería fallar, pero si puede ser demasiado lenta para el tiempo que tenemos, por lo que es necesario saber reconocer cuando es una buena idea utilizar esta técnica

6.2. Recursividad

La recursividad es una técnica de programación, en la que un módulo hace una llamada a si mismo con el fin de resolver el problema. La llamada a si mismo se conoce como llamada recursiva.

Dicho formalmente, un algoritmo se dice recursivo si calcula instancias de un problema en función de otras instancias del mismo problema hasta llegar a un caso base, que suele ser una instancia pequeña del problema, cuya respuesta generalmente está dada en el algoritmo y no es necesario calcularla.

Aún así la definición puede ser confusa, por eso ahora presentaremos un ejemplo que ilustra la recursividad:

Tenemos el caso del factorial, la notación del factorial esta dado por el símbolo ! así 5! se leerá "cinco factorial" o "factorial de 5", el factorial de un número n es:

$$\prod_{i=1}^n i \quad (6.1)$$

Es decir, el producto de todos los números de 1 hasta n . Por ejemplo 5!:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Ahora, si queremos ver el factorial como una recursión, podemos hacer uso de la propiedad asociativa:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

$$5! = 5 \times (4 \times 3 \times 2 \times 1)$$

18 Capítulo 6 Búsqueda Completa Recursiva

Si nos fijamos, todo lo que tenemos entre paréntesis es igual a 4!:

$$5! = 5 \times 4!$$

Y podemos seguir, ¿A que es igual 4!?

$$4! = 4 \times 3!$$

Llegaremos a un punto, donde tenemos 1! que es igual a 1 este sera nuestro caso base, entonces podemos definir el factorial de la siguiente forma:

$$f(x) = \begin{cases} \text{si } x = 1 & 1 \\ \text{si } x > 1 & x \times f(x-1) \end{cases} \quad (6.2)$$

El codigo de esto es el siguiente

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 int f (int x) {
4     if ( x == 1 ) return 1;
5     return x*f(x-1);
6 }
7 int main() {
8     int n;
9     cin>>n;
10    cout<<f(n)<<endl;
11    return 0;
12 }
```

6.2.1. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1262	Maximo Comun Divisor 2	Recursion	https://jv.umsa.bo/problem.php?id=1262

7

Sucesiones

7.1. Introducción

Una sucesión es un conjunto de cosas (normalmente números), que se listan una detrás de otra, en un cierto orden. Una sucesión es una aplicación cuyo dominio es el conjunto de los números naturales y su codominio es cualquier otro conjunto. Cada uno de ellos es denominado término (también elemento o miembro) de la sucesión y al número de elementos ordenados (posiblemente infinitos) se le denomina la longitud de la sucesión.

En general, este tipo de problemas se resumen en poder encontrar los siguientes términos de las sucesiones o los k términos siguientes, el problema tiene una solución mas analítica, encontrar las reglas con las que se generan los términos, para luego programarse, lo cual suele ser la parte más sencilla del problema. Ahora se verán algunos tipos de soluciones, que te ayudaran a encontrar estas reglas en sucesiones específicas.

7.2. Tipos de sucesiones

7.2.1. Sucesiones aritméticas

En una sucesión aritmética, cada término se calcula sumando un número fijo al anterior.

$$1, 2, 3, 4, 5, \dots$$

Esta sucesión tiene una diferencia de 1 entre cada término. La regla es $x_n = n$

$$3, 8, 13, 18, 23, \dots$$

Esta sucesión tiene una diferencia de 5 entre cada término. La regla es $x_n = n + 5$

7.2.2. Sucesiones geométricas

En una sucesión geométrica, cada término se calcula multiplicando el anterior por un número fijo.

$$2, 4, 8, 16, 32, \dots$$

Esta sucesión tiene un factor 2 entre cada término. La regla es $x_n = 2^n$

4, 2, 1, 0,5, 0,25, ...

Esta sucesión tiene un factor 0.5 (un medio) entre cada término. La regla es $x_n = 4 \times 2^{-n}$

7.2.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1488	Números pentagolanes	Sucesiones	https://jv.umsa.bo/problem.php?id=1488

8

Sucesión de Fibonacci

Existe una sucesión muy conocida, llamada Sucesión de Fibonacci, que es la siguiente

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 . . .

Puedes notar que la sucesión se forma de la siguiente manera:

$$x_n = x_{n-1} + x_{n-2}$$

Es decir, cada término de la sucesión es la suma de los dos anteriores, comenzando con los números 0 y 1. Esta sucesión es muy conocida, ya que tiene muchas aplicaciones en ciencias de la computación, matemática y teoría de juegos. También aparece en configuraciones biológicas, como por ejemplo, en las ramas de los árboles, en la disposición de las hojas en el tallo, en las flores de las alcachofas y girasoles, en las inflorescencias del brécol romanesco y en la configuración de las piñas de las coníferas. De igual manera, se encuentra en la estructura espiral del caparazón de algunos moluscos, como el nautilus.

Como se puede dar cuenta, esta sucesión crece muy rápido, en un entero de 32 bits solo puedes almacenar el término 47 de la sucesión.

Ahora, veremos algunas formas para generar los términos de esta serie:

8.1. Iterativo

La forma mas primitiva que se nos puede ocurrir, es tener dos variables para los términos en los que se iniciara la sucesión, e ir sumando y cambiando sus valores a medida que generamos los siguientes términos, suponiendo que queremos mostrar todos los términos de la sucesión hasta el término n — *símo*, el código sería el siguiente:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int a=-1, b=1;
5      int n;
6      cin>>n;
7      for(int i=0; i<n; i++){
```

```

8         int f=a+b;
9         cout<<f<<endl;
10        a=b;
11        b=f;
12    }
13    return 0;
14 }

```

En este caso iniciamos los valores en -1 y 1 , para que empezar a generar el 0 (que es el primer término de la sucesión), también se podría iniciar los valores con 0 y 1 , y generar desde el segundo término de la sucesión.

8.2. Recursivo

Como se puede ver en la definición del principio, la sucesión puede verse como una recursión:

$$f(x) = \begin{cases} \text{si } x > 2 & f(x-1) + f(x-2) \\ \text{si } x = 2 & 1 \\ \text{si } x = 1 & 0 \end{cases} \quad (8.1)$$

El código para calcular el n – simo valor de la sucesión, de manera recursiva, es el siguiente:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  int f (int n) {
4      if ( n == 1 ) return 0;
5      if ( n == 2 ) return 1;
6      return f(n-1)+f(n-2);
7  }
8  int main() {
9      int n;
10     cin>>n;
11     cout<<f(n)<<endl;
12     return 0;
13 }

```

8.2.1. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1342	Fibonacci Modular	Fibonacci	https://jv.umsa.bo/problem.php?id=1342

9

Arreglos Unidimensionales

9.1. Introducción

Un arreglo es una colección de datos del mismo tipo, que se almacenan en posiciones consecutivas de memoria y reciben un nombre común. Un arreglo tiene una colección finita y homogénea de elementos. **Finita:** Todo arreglo tiene un límite; es decir, debe determinarse cuál será el número máximo de elementos que podrán formar parte del arreglo. **Homogénea:** Todos los elementos del arreglo deben ser del mismo tipo.

9.1.1. Recorrido de un vector

Para leer un vector estático vamos a utilizar los métodos básicos de entrada de datos de C++

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4  int v[100]; //definimos un arreglo estatico con
5              //un maximo de 100 elementos
6              //numerados de 0 ... 99
7
8  int main(){
9      int n;
10     cin>>n;
11     for(int i = 0; i < n; i++){
12         cin>>v[i]; // leemos cada posicion del arreglo
13     }
14
15     for(int i = 0; i < n; i++){
16         cout<<v[i]<<" "; //imprimimos cada posicion del arreglo
17     }
18     cout<<endl;
19
20     return 0;
21 }
```

9.1.2. Búsqueda de un elemento en un arreglo

Para realizar la búsqueda de un elemento dentro de un arreglo unidimensional, simplemente necesitamos recorrer el arreglo con una estructura de control que nos permita iterar el mismo,

recordando siempre que los elementos están numerados desde 0 hasta $n - 1$ donde n es el numero de elementos en el arreglo

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  int v[100]; //definimos un arreglo estatico con un maximo de 100 elementos
5              //numerados de 0 ... 99
6
7  int main(){
8      int n,x;
9      cin>>n;
10     for(int i = 0; i < n; i++){
11         cin>>v[i]; // leemos cada posicion del arreglo
12     }
13     cout<<"Introduzca el elemento a buscar"<<endl;
14     cin>>x;
15     bool encontrado = false;
16     for(int i = 0; i < n; i++){
17         if(v[i] == x){
18             cout<<"el elemento se encuentra en la
19                 posicion "<<i<<endl;
20             encontrado = true;
21         }
22     }
23     if(!encontrado)
24         cout<<"el elemento "<<x<<" no fue encontrado en
25         el arreglo"<<endl;
26     return 0;
27 }
```

9.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1406	Promedios	Arreglos	https://jv.umsa.bo/problem.php?id=1406
1294	Suma en rango	Arreglos	https://jv.umsa.bo/problem.php?id=1294

10

Ordenamiento de Arreglos

10.1. Ordenamiento por burbuja

El algoritmo de ordenamiento burbuja o bubble sort en inglés, es el algoritmo más fácil de aprender y corto de programar, trata de comparar cada elemento con el resto de los demás.

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4  int v[100];
5
6  int main(){
7      int n,x;
8      cin>>n;
9      for(int i = 0; i < n; i++){
10         cin>>v[i];
11     }
12     for(int i = 0; i < n; i++){
13         for(int j = i + 1; j < n; j++){
14             if(v[j] < v[i]){
15                 swap(v[i],v[j]); //metodo para
                                intercambiar el valor de 2 variables
16             }
17         }
18     }
19     for(int i = 0; i < n; i++){
20         cout<<v[i]<<" ";
21     }
22     cout<<endl;
23     return 0;
24 }
```

10.1.1. Ordenamiento por inserción

Imaginemos que hay k elementos ordenados de menor a mayor, se toma el elemento $k+1$ y se compara con todos los elementos ya ordenados, deteniéndose cuando se encuentra un elemento menor (todos los elementos mayores han sido desplazados una posición a la derecha) o cuando ya no se encuentran elementos (todos los elementos fueron desplazados y este es

el más pequeño). En este punto se inserta el elemento $k + 1$ debiendo desplazarse los demás elementos, cuando $k = 1$ entonces el arreglo esta ordenado.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  int A[100];
5
6  int main(){
7      int n,x;
8      cin>>n;
9      for(int i = 0; i < n; i++){
10         cin>>A[i];
11     }
12     int j,v;
13     for (int i = 1; i < n; i++) {
14         v = A[i];
15         j = i - 1;
16         while (j >= 0 && A[j] > v) {
17             A[j + 1] = A[j];
18             j--;
19         }
20
21         A[j + 1] = v;
22     }
23
24     for(int i = 0; i < n; i++){
25         cout<<A[i]<<" ";
26     }
27     cout<<endl;
28     return 0;
29 }
```

10.1.2. Ordenamiento por selección

Este algoritmo de ordenamiento selecciona el elemento mas pequeño en el arreglo y lo coloca al principio, se repite ese proceso obviando los elementos previamente seleccionados.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  int A[100];
5
6  int main(){
```

```

7   int n,x;
8   cin>>n;
9   for(int i = 0; i < n; i++){
10      cin>>A[i];
11  }
12  int min,aux;
13  for (int i=0; i<n-1; i++) {
14      min=i;
15      for(int j = i + 1 ; j < n ; j++)
16          if(A[min] > A[j])
17              min=j;
18      aux=A[min];
19      A[min]=A[i];
20      A[i]=aux ;
21  }
22
23  for(int i = 0; i < n; i++){
24      cout<<A[i]<<" ";
25  }
26  cout<<endl;
27  return 0;
28  }

```

10.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1028	Contando en sort de Inserción	Ordenamiento	https://jv.umsa.bo/problem.php?id=1028
1058	Ordenando números	Ordenamiento	https://jv.umsa.bo/problem.php?id=1058

11

Ordenamiento en C++

11.1. Introducción

Como la mayoría de los lenguajes, el C++ también tiene métodos ya implementados en sus librerías para ordenar arreglos.

11.1.1. Ordenamiento de un arreglo estático

Ya sabemos como se instancia un arreglo estático y ahora vamos a aprender a ordenarlo con librerías del C++

```
1
2  #include <bits/stdc++.h>
3
4  using namespace std;
5  int v[100];
6
7  int main(){
8      int n,x;
9      cin>>n;
10     for(int i = 0; i < n; i++){
11         cin>>v[i];
12     }
13     sort(v,v + n);
14     for(int i = 0; i < n; i++)
15         cout<<v[i]<<" ";
16     cout<<endl;
17     return 0;
18 }
```

11.1.2. Ordenamiento de un arreglo dinámico

Para ordenar un arreglo de tipo vector, también utilizaremos el método **sort** del C++

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  vector<int> v;
```

```

6
7  int main(){
8      int n,x;
9      cin>>n;
10     v.assign(n,0); // asignamos el tamaño n y los rellenos de 0
11     for(int i = 0; i < n; i++){
12         cin>>v[i];
13     }
14     sort(v.begin(),v.end());
15     for(int i = 0; i < n; i++){
16         cout<<v[i]<<" ";
17     }
18     cout<<endl;
19     return 0;
20 }

```

11.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1333	Easy Suffix Array	Librería de Ordenar	https://jv.umsa.bo/problem.php?id=1333
1029	Escoger Equipos	https://jv.umsa.bo/problem.php?id=1029	

12

12.1.

Permutaciones en C++

Introducción

Una permutación es la variación del orden o de la disposición de los elementos en un conjunto, para este caso particular manejaremos nuestro conjunto de elementos en un arreglo.

Por ejemplo, en el conjunto 1, 2, 3, cada ordenación posible de sus elementos, sin repetirlos, es una permutación. Existe un total de 6 permutaciones para conjuntos de 3 elementos, en este caso: "1, 2, 3", "1, 3, 2", "2, 1, 3", "2, 3, 1", "3, 1, 2", "3, 2, 1".

El poder calcular todas las permutaciones de un conjunto de datos, nos puede ayudar a resolver ejercicios.

12.1.1. Permutación de un arreglo estático

Para poder permutar un arreglo en C++, como pre requisito necesitamos que nuestro arreglo este ordenado, ya que si no es así, entonces no podremos hallar todas las permutaciones.

```
1
2  #include <bits/stdc++.h>
3
4  using namespace std;
5  int v[100];
6
7  int main(){
8      int n,x;
9      cin>>n;
10     for(int i = 0; i < n; i++){
11         cin>>v[i];
12     }
13     sort(v,v + n);
14     do{
15
16         for(int i = 0; i < n; i++)
17             cout<<v[i]<<" ";
18         cout<<endl;
19
20     }while(next_permutation(v,v + n));
21     return 0;
22 }
```

12.1.2. Permutación de un arreglo dinámico

Lo propio, para poder hallar todas las permutaciones en un arreglo dinámico, se tiene que ordenar previamente el arreglo.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4  vector<int> v;
5
6  int main(){
7      int n,x;
8      cin>>n;
9      v.assign(n,0);
10     for(int i = 0; i < n; i++){
11         cin>>v[i];
12     }
13     sort(v.begin(),v.end());
14     do{
15
16         for(int i = 0; i < n; i++)
17             cout<<v[i]<<" ";
18         cout<<endl;
19
20     }while(next_permutation(v.begin(),v.end()));
21     return 0;
22 }

```

12.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1032	Hola Mundo!	Impresion	https://jv.umsa.bo/problem.php?id=1232
1425	Imprimir PI E Au	Impresion	https://jv.umsa.bo/problem.php?id=1425

13

Números Primos

13.1. Introducción

Se define a un número primo, como un número natural mayor a 1 que solo tiene dos distintos divisores, el 1 y el mismo numero.

13.1.1. Test de primalidad

Para saber si un número es primo o no, se puede programar un algoritmo mediante divisiones sucesivas, la idea es revisar entre 1 y N cuantos números dividen a N , si se encuentran solo 2 divisores entonces decimos que N es primo, caso contrario N es un numero compuesto.

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool esPrimo(int n){
6      int divisores = 0;
7      for(int i = 1; i <= n; i++){
8          if(n % i == 0)
9              divisores++;
10     }
11     if(divisores == 2)
12         return true;
13     return false;
14 }
15
16 int main(){
17     int n;
18     cin>>n;
19     if(esPrimo(n)){
20         cout<<"El numero "<<n<<" es numero primo"<<endl;
21     }else{
22         cout<<"El numero "<<n<<" NO es primo"<<endl;
23     }
24     return 0;
25 }
```

13.1.2. Números primos mediante divisiones sucesivas

Para hallar la serie de los números primos, podemos utilizar el anterior algoritmo, con una simple modificación, sin embargo es tarea del lector ver hasta que punto es óptimo dicho algoritmo.

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool esPrimo(int n){
6      int divisores = 0;
7      for(int i = 1; i <= n; i++){
8          if(n % i == 0)
9              divisores++;
10     }
11     if(divisores == 2)
12         return true;
13     return false;
14 }
15
16 int main(){
17
18     for(int i = 1; i <= 100; i++){
19         if(esPrimo(i))
20             cout<<i<<" ";
21     }
22     cout<<endl;
23     return 0;
24 }

```

13.1.3. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1462	Matriz Prima	Primos	https://jv.umsa.bo/jv/problem.php?id=1462

14 Manejo y Procesamiento de Cadenas

14.1. Introducción

En C++ podemos manejar las cadenas de dos formas, como un vector de caracteres o como un objeto de tipo string, a continuación detallaremos el manejo como un objeto de tipo string.

14.1.1. String en C++

La creación de un objeto string en C++, se realiza de la siguiente manera:

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      string s;
7      cout<<"Introduzca su nombre: "<<endl;
8      cin>>s;
9      cout<<"Hola: "<<s<<endl;
10     return 0;
11 }
```

14.1.2. Operación de concatenación

La concatenación de cadenas en C++, se refiere a la operación de colocar una cadena a continuación de otra, seguidamente vamos a utilizar el anterior código, concatenando la palabra "Hola: " antes del nombre del usuario.

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      string s;
7      cout<<"Introduzca su nombre: "<<endl;
8      cin>>s;
9      s = "Hola: " + s;
```

```

10     cout<<s<<endl;
11     return 0;
12 }

```

14.1.3. Métodos de la clase String en C++

A continuación, detallaremos los métodos mas usados de la clase String en C++

```

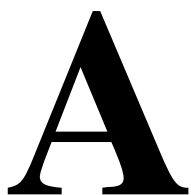
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main(){
6      string s = "Hola mundo";
7      int tam = s.size(); // obtenemos el tamaño de la cadena s
8      cout<<"El tamaño es: "<<tam<<endl;
9
10     char x = 'Y';
11     s = s + x; // concatenamos un caracter a la cadena
12     cout<<"La nueva cadena es: "<<s<<endl;
13
14     string str="Aprendiendo los metodos de manejo de cadenas.";
15
16     string str2 = str.substr (3,5);      // .end()
17
18     int pos = str.find("metodos");      // posición de "metodos" en str
19
20     string str3 = str.substr (pos);      // cortar "metodos" hasta el
        final
21
22     cout<<"La cadena cortada es: "<<str2<<endl;
23     cout << str3 << '\n';
24     return 0;
25 }

```

14.1.4. Ejercicios

Para poner en práctica lo aprendido, siga el siguiente enlace:

Juez Patito	Nombre	Tipo de problema	Enlace
1297	Invertir palabras 1	Cadenas	https://jv.umsa.bo/problem.php?id=1297
1296	Invertir palabras 1	Cadenas	https://jv.umsa.bo/problem.php?id=1296



Lista de Ejercicios

Juez Patito	Nombre	Tipo de problema	Enlace
1032	Hola Mundo!	Impresion	https://jv.umsa.bo/problem.php?id=1232
1425	Imprimir PI E Au	Impresion	https://jv.umsa.bo/problem.php?id=1425
1226	Temperatuta	Condicionales	https://jv.umsa.bo/problem.php?id=1226
1248	Números Armónicos	Ciclos For	https://jv.umsa.bo/problem.php?id=1248
1249	Primeros números	Ciclos For	https://jv.umsa.bo/problem.php?id=1249
1065	Kuadraconia	Adhoc	https://jv.umsa.bo/problem.php?id=1065
1038	La clase estaba aburrida	Busqueda Completa	https://jv.umsa.bo/problem.php?id=1038
1262	Maximo Comun Divisor 2	Recursion	https://jv.umsa.bo/problem.php?id=1262
1488	Números pentagolanes	Sucesiones	https://jv.umsa.bo/problem.php?id=1488
1342	Fibonacci Modular	Fibonacci	https://jv.umsa.bo/problem.php?id=1342
1406	Promedios	Arreglos	https://jv.umsa.bo/problem.php?id=1406
1294	Suma en rango	Arreglos	https://jv.umsa.bo/problem.php?id=1294
1028	Contando en sort de Inserción	Ordenamiento	https://jv.umsa.bo/problem.php?id=1028
1058	Ordenando números	Ordenamiento	https://jv.umsa.bo/problem.php?id=1058
1333	Easy Suffix Array	Librería de Ordenar	https://jv.umsa.bo/problem.php?id=1333
1029	Escoger Equipos	Librería de Ordenar	https://jv.umsa.bo/problem.php?id=1029
1032	Matriz Prima	Primos	https://jv.umsa.bo/jv/problem.php?id=1462
1032	Hola Mundo!	Permutaciones	https://jv.umsa.bo/problem.php?id=1232
1297	Invertir palabras 1	Cadenas	https://jv.umsa.bo/problem.php?id=1297
1296	Invertir palabras 1	Cadenas	https://jv.umsa.bo/problem.php?id=1296

List of Algorithms

1.	Condicionales simples	5
2.	Condicionales dobles	6
3.	Ciclo For	9
4.	While	10